LLM Inference Fundamentals

pig7selene

September 5, 2026

Abstract

A trained decoder-only language model generates by repeatedly converting a prefix into next-token logits, selecting one token, and extending the prefix. This chapter separates prompt Prefill from incremental Decode, derives common sampling rules from logits, and explains the role and memory cost of the KV Cache. It then introduces the serving metrics that distinguish an interactive system's first-token latency, per-token generation latency, and aggregate throughput. The goal is a precise execution model for later chapters on cache management and serving optimization.

1 Introduction

Pretraining teaches a decoder-only Transformer to assign conditional probabilities to token sequences. At inference time, the same computation is no longer evaluated against a known continuation. A serving system receives a prompt, uses the model to form a distribution for the next token, selects one token according to a decoding rule, and feeds that selected token back as part of the next prefix. The resulting dependence makes generation sequential even when the prompt itself can be processed with substantial parallelism.

Chapter 20 closed the Post-training section by evaluating model behavior under a declared generation protocol. The Inference and Serving section now turns to the execution path that realizes such a protocol. Chapter 5 derived the Causal Language Modeling objective, and Chapter 3 defined causal Self-Attention, key-value heads, and their cached tensors. Here those pieces are treated as an inference program. The discussion stops before cache virtualization, quantization, speculative decoding, continuous batching, and distributed serving. Those techniques optimize the execution model established here; they do not replace it.

Let $\mathcal{V}$ be the vocabulary with size $V = |\mathcal{V}|$. A prompt is a sequence of token IDs $x = (x_1, \ldots, x_S)$, where S is its current length. The model parameters θ are fixed during ordinary inference. When the model has processed the current prefix, it produces a vocabulary-logit vector $z_t \in \mathbb{R}^V$ for the next generated position t. A decoding rule converts z_t into either a deterministic choice or a random draw.

2 From Training to Inference

The training and inference computations share the same conditional distribution, but they use it for different purposes. In teacher-forced training, the corpus supplies every token of a complete sequence. The model can therefore score many next-token targets in parallel, subject to the causal Attention Mask. The loss measures the probability assigned to observed targets; Chapter 5 shows how those token-level terms aggregate into the negative log-likelihood.

At inference time, a future target is unavailable. After the model forms $p_{\theta(\cdot \mid x_1, \ldots, x_S)}$, a decoding policy must select a token $\hat{x}_{S+1}$. That token becomes part of the context for the next forward step. For a generated continuation $y = (y_1, \ldots, y_N)$, this feedback loop has the form

$$y_t \sim q\big(\cdot \mid x, y_{\{<t\}}\big), \quad y_{\{<t\}} = \big(y_1, \ldots, y_{\{t-1\}}\big), \tag{1}$$

where q is the effective decoding distribution. It may equal p_θ, or it may be a transformed and truncated version of the model distribution. The distinction is important: the language model defines logits and conditional probabilities, whereas a serving policy defines how to choose among them. A change in temperature or sampling threshold can change generated text without changing θ.

The loop terminates when it selects an end-of-sequence token, reaches a declared output-token limit, or satisfies an application-level stop condition. A stop string is not normally a token-level property of the probability model. It is a rule applied to decoded output, often after checking a token sequence for one of several configured suffixes. The serving contract must specify whether the stop tokens themselves are returned and whether they remain in the internal context.

3 Prompt Processing: Prefill and Decode

Inference has two execution regimes. The first is **Prefill**, also called prompt processing. Given all S prompt tokens, the system runs the Transformer over the complete prompt, constructs the cache needed by future attention layers, and obtains logits for the first output token:

$$(x_1, ..., x_S) \to (K_1, V_1, ..., K_L, V_L, z_0). \tag{2}$$

Here L is the number of Transformer blocks, and (K_l, V_l) denotes the stored key and value tensors for layer l. Causal masking preserves the autoregressive dependency rule, but all prompt positions are present, so the projections, attention computations, and Feed-Forward Networks can operate on many token positions concurrently. For long prompts, this stage can contain much more arithmetic work than one decoding step.

After the first selected token is appended, the system enters **Decode**. Instead of reprocessing the entire prefix, the new token is embedded and passed through one incremental Transformer step. At each layer, its new query attends to cached keys and values from the prior prefix, while its new key and value are appended to the cache:

$$((K_l, V_l), y_t) \to (K_l^+, V_l^+, z_t), \tag{3}$$

where the superscript $+$ denotes the cache after the new token has been incorporated. Sampling from z_t produces y_{t+1}, which triggers the next step. Autoregressive generation is therefore sequential over output tokens, not necessarily over the input prompt. Large Transformer inference studies distinguish the high-parallelism prefill workload from the less parallel incremental-generation workload for precisely this reason [1].

Phase	Input available at once	Persistent state produced or consumed	Primary user-visible consequence
Prefill	All prompt tokens for each request.	Constructs keys and values for every prompt position; produces first-token logits.	Dominates Time to First Token for a long prompt.
Decode	One newly selected token per active request per step.	Reads the prefix KV Cache and appends one layerwise key-value pair.	Determines incremental token delivery rate.

Table 1: Prefill and Decode use the same parameters but expose different shapes and bottlenecks. Prefill processes a known token block; Decode extends an unknown continuation one token at a time.

3.1 The KV Cache

Without a KV Cache, decoding token y_t would require recomputing the keys and values of every earlier token at every layer. The cache preserves those earlier projections. The current token still needs a query, key, and value projection, but attention can compare its one new query against stored keys and combine stored values rather than regenerating the prefix's projections. This avoids redundant prefix work and makes incremental decoding practical.

Use the attention notation of Chapter 3: g is the number of key-value heads and d_h is head dimension. For a batch of B active sequences with current length S, one layer stores

$$K_l, V_l \in \mathbb{R}^{B \times g \times S \times d_h}. \tag{4}$$

If every cached scalar uses b_{cache} bytes, the cache footprint for all layers is approximately

$$M_{\text{KV}} = 2BLSgd_h b_{\text{cache}}. \tag{5}$$

The factor two accounts for keys and values. This accounting excludes model weights, activations and temporary workspaces, allocator overhead, and padding or fragmentation. It nevertheless exposes the central scaling law: cache memory grows linearly with sequence length, batch size, layer count, key-value head count, head dimension, and bytes per cached scalar. Grouped-Query Attention and Multi-Query Attention reduce g, which Chapter 3 explains as an architectural trade-off that is especially valuable during decoding.

Model-weight memory and KV Cache memory are distinct. Weight memory is largely fixed for a loaded model and depends on parameter count and weight dtype. KV Cache memory is request-dependent state. It grows while a request generates, varies across prompts and stopping times, and limits how many long requests can coexist. Production serving research identifies this dynamically sized cache as a principal source of memory pressure under batched generation [2].

4 From Logits to Tokens

The final hidden state at the current position is mapped by the language-model head to logits $z_t = (z_{t,1}, ..., z_{t,V})$. As in Chapter 5, Softmax converts those unnormalized scores to a categorical distribution. In inference, however, this distribution is an input to a decision rule rather than to cross-entropy with a known target.

4.1 Greedy Decoding and Temperature

Greedy decoding chooses the most probable token at every step:

$$y_{t+1} = \text{argmax}_{i \in \{1,...,V\}} z_{t,i}. \tag{6}$$

Because Softmax preserves the ordering of logits, taking an argmax over logits is equivalent to taking one over probabilities. Greedy decoding is deterministic if the model, tokenizer, prompt formatting, and numerical execution are deterministic. It is often useful when exact reproducibility or a constrained output format matters, but it follows only one local continuation and does not optimize the probability of a complete sequence globally.

Temperature changes the sharpness of the categorical distribution before sampling. For $T_{\text{temp}} > 0$,

$$p_{t,i}^{T_{\text{temp}}} = \text{softmax}_{i\left(\frac{z_{t,i}}{T_{\text{temp}}}\right)}. \tag{7}$$

When $T_{\text{temp}} = 1$, this is the model's ordinary Softmax distribution. As T_{temp} approaches zero, probability concentrates on the largest logit and sampling approaches greedy selection; implementations handle this limiting case by using greedy decoding rather than dividing by zero. Values above one flatten the distribution and increase the relative probability of lower-ranked tokens. Temperature does not add knowledge or correct a model error. It changes how readily the system explores alternatives already represented in its logits.

4.2 Top-k and Top-p Sampling

Unrestricted sampling can select tokens from a long, low-probability tail. **Top-k sampling** first retains only the k highest-probability tokens. Let $K_{k(z_t)}$ be their index set. Its truncated distribution is

$$q_{t,i} = \begin{cases} \frac{p_{t,i}}{\sum_{j \in K_{k(z_t)}} p_{t,j}} i \in K_{k(z_t)} \\ \backslash\, 0 \qquad\qquad \text{otherwise.} \end{cases} \tag{8}$$

and y_{t+1} is sampled from q_t. The cardinality is fixed, but the probability mass covered by the retained set can vary widely across contexts. Top-k sampling has been used in open-ended neural text generation as a practical truncation rule [3].

Top-p sampling, or nucleus sampling, chooses the smallest prefix of tokens sorted by decreasing probability whose cumulative mass reaches a threshold p_{nucleus}. If $P_{p_{\text{nucleus}}}(z_t)$ denotes that set, the sampling distribution is renormalized over $P_{p_{\text{nucleus}}}(z_t)$ exactly as in Equation 8. The retained set has variable size: a peaked distribution may require few tokens, whereas a diffuse distribution may require

many. Holtzman et al. motivate this adaptive truncation by observing that the low-probability tail of an open-ended language-model distribution can be unreliable for generation [4].

Temperature, Top-k, and Top-p are not interchangeable. Temperature reshapes every probability before a later filter; Top-k fixes a candidate count; Top-p fixes a retained probability mass. Their composition is an implementation decision. A reproducible serving interface must record the order of operations, thresholds, random seed policy, and whether any token-specific penalties were applied before sampling.

4.3 Repetition Controls, Stops, and Beam Search

Practical interfaces often transform selected logits based on the generated prefix. A repetition penalty, frequency penalty, or presence penalty changes the relative scores of tokens that have already occurred. Such controls may reduce a visible repetition failure, but they change the effective distribution q in Equation 1 and can also suppress legitimate repetition. They should be regarded as decoding-policy parameters, not learned properties of the base model.

Stopping is likewise part of the policy layer. A request typically has a maximum output-token budget, one or more end-of-sequence or application stop sequences, and perhaps a context-window limit. A system should check a stop condition after selecting a token and before scheduling another Decode step. If a generated token would exhaust the allowed context, the request must stop or follow a separately declared context-management policy; silently exceeding the model's positional contract is not valid inference.

Beam Search keeps several partial continuations and repeatedly expands the most promising ones under cumulative log-probability. It is useful when a task admits a short, sharply scored output and a sequence-level search budget is appropriate. For open-ended LLM interaction, however, deterministic likelihood maximization can produce generic or repetitive text, and stochastic sampling is often more central. This is a task-dependent observation, not a rule that Beam Search is universally inappropriate; empirical work on neural text degeneration makes the distinction explicit [4]. Beam Search also multiplies active hypotheses and hence cache state, so its systems cost differs from a single sampled continuation.

5 Context, Batching, and Serving Metrics

The **context length** of an active request is the number of tokens currently available to the Transformer: prompt tokens plus generated tokens, together with any required special tokens. It is not only a model-side maximum. It affects prefill work, attention reads during Decode, and the cache footprint in Equation 5. A request with a short output can still have a high first-token cost if its prompt is long; a request with a short prompt can become expensive over a long continuation because every new token enlarges the prefix read by later decoding steps.

Batch inference processes multiple independent requests together. During Prefill, batching combines prompt tokens from multiple requests to expose larger matrix operations. During Decode, a scheduler may advance several active requests in one step, usually after padding or otherwise representing their differing current lengths. The batch size B in Equation 5 is therefore an operational quantity: increasing it can improve hardware utilization and total throughput, but it also increases cache memory and may delay an individual request while it waits to be admitted or scheduled.

Metric	Typical unit	Interpretation
Time to First Token (TTFT)	seconds or milliseconds	Elapsed time from a request's arrival or dispatch, under a declared convention, until its first generated token is available. It includes queueing and Prefill under many service definitions.
Time per Output Token (TPOT)	milliseconds per token	Incremental time between delivered generated tokens after the first. It reflects Decode scheduling and execution, not only model arithmetic.
Output rate	output tokens per second	The reciprocal of average TPOT under a compatible measurement convention.
Request rate	requests per second	Completed or admitted requests per unit time; it must be reported with prompt and output-length distributions.
Token throughput	tokens per second	Aggregate input and/or output tokens processed per wall-clock time. The included token types must be declared.

Table 2: Serving metrics answer different questions. A high aggregate token throughput does not by itself imply low interactive latency, and a low TTFT does not guarantee rapid subsequent tokens.

For one request with N generated tokens, end-to-end latency contains queueing time, Prefill time, and the accumulated time of its Decode steps. TTFT is closely related to the first two components and the first Decode selection, depending on the service boundary. TPOT summarizes the later Decode work. A throughput-oriented workload may choose a larger batch and tolerate more queueing; an interactive workload usually imposes a tighter TTFT and TPOT target. The same model can therefore have different preferred schedules for different latency objectives.

Prefill is often compute-heavy because it processes many known token positions and can use large matrix operations efficiently. Decode has much less token-position parallelism for a single request. It repeatedly reads model weights and a growing KV Cache while producing a small amount of new-token work, so its realized performance is frequently limited by memory bandwidth or memory access rather than by peak floating-point throughput. This is a workload-level tendency, not a hardware-independent law: model shape, batch size, context length, dtype, kernels, and device topology all matter [1]. Later chapters will examine the optimization techniques that change this trade-off.

6 Implementation Contracts

An inference implementation should make the model-data interface as explicit as a training implementation. The tokenizer, special tokens, and Chat Template used to construct the prompt must match the model's expected convention; Chapter 13 explains why formatting is part of the learned interface for chat models. The prompt must fit the declared context policy before Prefill, and the token selected at each Decode step must be appended exactly once before the next model call.

The cache needs a precise ownership and shape contract. For each request and layer, keys and values must have the dtype, head mapping, sequence positions, and batch association expected by the attention kernel. A cache may not be reused across unrelated requests unless an explicit prefix-sharing rule establishes that the corresponding token prefix, model revision, positional convention, and cache representation are identical. When a request stops or is cancelled, its cache allocation must become unavailable to future reads.

The decoding contract should record the complete policy: greedy or sampled mode; temperature; Top-k and Top-p thresholds; repetition controls; random-number-generator and seed behavior; maximum tokens; stop rules; and whether terminal tokens are emitted. Tests on a small model should compare

cached Decode logits with logits from a full recomputation on the same prefix, up to the numerical tolerance of the execution dtype. They should also verify that an identical deterministic policy yields identical tokens, that a fixed seeded sampling policy is reproducible under a declared runtime configuration, and that TTFT, TPOT, and throughput are measured with stated start and end events.

7 Summary

Inference turns the conditional distribution learned in pretraining into a sequential execution loop. Prefill processes a known prompt in parallel where possible, constructs the layerwise KV Cache, and produces first-token logits. Decode then adds one selected token at a time, reusing cached keys and values so that earlier prefix projections are not recomputed. The model supplies logits; temperature, truncation, penalties, and stop rules define the serving policy that turns those logits into an output sequence.

KV Cache memory is dynamic request state, distinct from fixed model-weight memory. Its linear dependence on active batch size, sequence length, layers, key-value heads, head dimension, and dtype explains why inference capacity and latency cannot be understood from parameter count alone. TTFT, TPOT, and throughput capture complementary consequences of Prefill, Decode, queueing, batching, and memory traffic. These concepts establish the baseline for the cache-management and execution optimizations considered next.

References

[1] R. Pope *et al.*, "Efficiently Scaling Transformer Inference," *arXiv preprint arXiv:2211.05102*, 2022, doi: 10.48550/arXiv.2211.05102.

[2] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, Association for Computing Machinery, 2023, pp. 611–626. doi: 10.1145/3600006.3613165.

[3] A. Fan, M. Lewis, and Y. Dauphin, "Hierarchical Neural Story Generation," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2018, pp. 889–898. doi: 10.18653/v1/P18-1082.

[4] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The Curious Case of Neural Text Degeneration," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=rygGQyrFvH